App Dev Project Report

1. Student Details

Name: Prabhjot Kaur Batth

Roll Number: DS23f2004695

Email id: 23f2004695@ds.study.iitm.ac.in

About Me: I am a student of IIT Madras and am undertaking my App Dev project that is a Hospital Management System. My fields of interest include web development and real-world problems and how technology can be practically applied in solving them.

2. Project Details

Project Title: Hospital Management System V2

Problem Statement:

To design and build a web-based application that streamlines hospital operations and improves patient care by enabling efficient management of doctors, patients, appointments, and treatment records.

Approach:

The application is built on Flask for the backend REST API and Vue.js 3 (CDN) for the frontend SPA.

Key features include:

Admin, Doctor, and Patient role-based authentication using JWT tokens.

Doctor, patient, and appointment CRUD operations.

Centralized medical records and treatment documentation.

Doctor scheduling and availability management for the next 7 days.

Async background jobs via Celery and Redis for reminders, reports, and CSV export.

The application follows an MVC-style architecture. Models, API routes, templates, and static files are clearly separated.

3. AI/LLM Declaration

Areas of AI Assistance:

- Writing SQLAlchemy model definitions and reviewing relationship configurations.
- Understanding Vue.js 3 CDN component patterns, props, and emit syntax.
- Generating boilerplate code structures for Flask REST endpoints.
- Creating API documentation samples and docstrings.
- Improving variable naming consistency across the codebase.
- Drafting the HTML structure for the monthly report template in tasks.py.

Manual Implementation:

- All core business logic and algorithms.
- Database schema design and entity relationships.

- API endpoint implementation and validation logic.
- JWT token management and role-based access control.
- Frontend Vue.js component organization, state management (store.js), and routing.
- Celery task design and Redis integration for background jobs.
- Debugging, testing, and end-to-end integration.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework for routing and request handling
Flask-JWT-Extended	JWT token-based authentication and role-based access control
SQLAlchemy / Flask-SQLAlchemy	ORM for SQLite database interaction
Vue.js 3 (CDN)	Reactive SPA frontend with modular components
Bootstrap 5 (CDN)	Frontend styling and responsive design
SQLite	Lightweight local database for storing all application data
Redis	Message broker for Celery background tasks
Celery	Async task queue for background jobs
Flask-Mail	Email delivery for reports and notifications
Werkzeug	Password hashing and security utilities
Jinja2	Template engine for entry-point HTML rendering only
Python	Core backend programming language

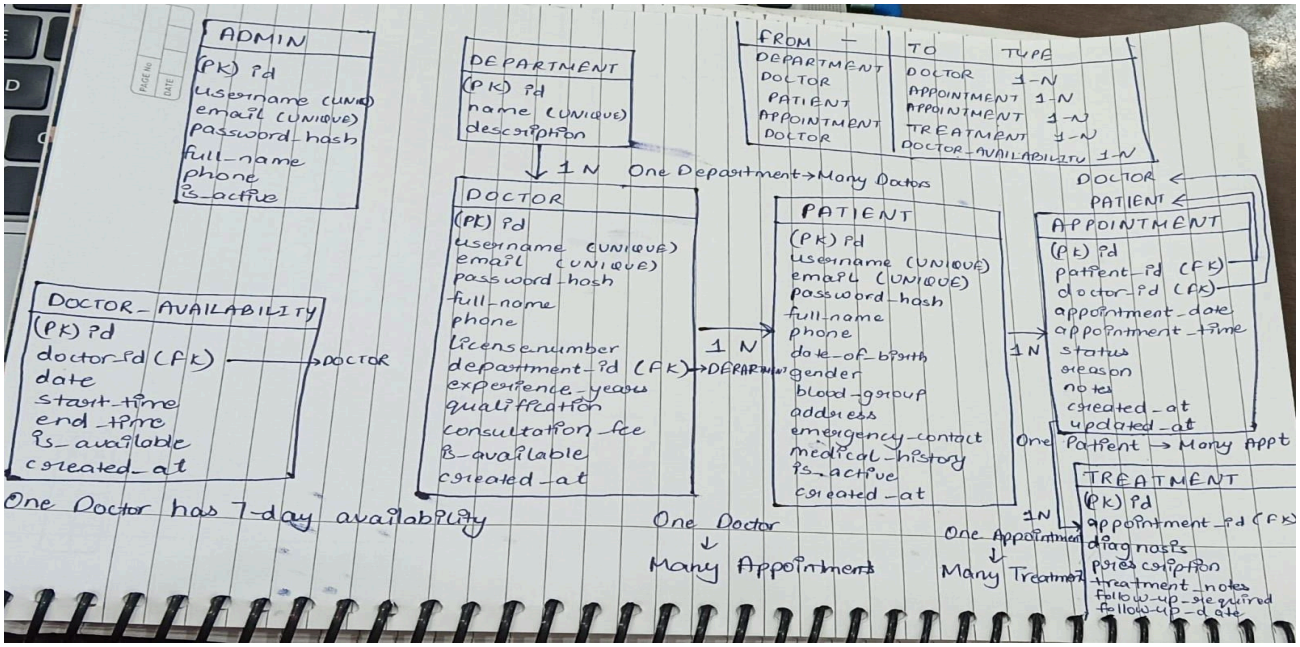
5. Database Schema / ER Diagram

Tables:

1. **Admin Table**
   Stores admin user credentials and profile information
2. **Doctor Table**
   Medical professional records with specialization and availability
3. **Patient Table**
   Patient records with medical information
4. **Department Table**
   Medical specializations/departments
5. **Appointment Table**
   Consultation bookings linking patients and doctors
6. **Treatment Table**
   Medical treatment records for completed appointments
7. **DoctorAvailability Table**
   Weekly schedule for doctor availability

Relationships:

From	To	Type	Description
users	doctors	1 → 1	One user account per doctor
users	patients	1 → 1	One user account per patient
departments	doctors	1 → N	One department has many doctors
doctors	appointments	1 → N	One doctor has many appointments
patients	appointments	1 → N	One patient has many appointments
appointments	treatments	1 → 1	One appointment has one treatment record
doctors	doctor_availability	1 → N	One doctor has many availability slots



[DB Schema]

6. API Resource Endpoints

Endpoint	Method	Role	Description
/api/auth/login	POST	All	Authenticate user and receive JWT token
/api/auth/register	POST	Public	Register new patient account
/api/departments	GET	All	List all specialization departments
/api/admin/doctors	GET/POST	Admin	List all doctors / Add new doctor
/api/admin/doctors/<id>	PUT/DELETE	Admin	Update or remove a doctor
/api/admin/patients	GET	Admin	List all patients
/api/admin/patients/<id>	PUT	Admin	Update or blacklist a patient
/api/admin/appointments	GET	Admin	View all appointments
/api/admin/stats	GET	Admin	Dashboard statistics (cached)
/api/doctor/profile	GET/PUT	Doctor	View / update own profile
/api/doctor/appointments	GET	Doctor	Get assigned appointments

/api/doctor/appointments/<id>	PUT	Doctor	Mark appointment complete or cancel
/api/doctor/treatments/<id>	POST	Doctor	Add treatment record to appointment
/api/doctor/availability	GET/POST	Doctor	View / set availability for next 7 days
/api/patient/profile	GET/PUT	Patient	View / update own profile
/api/patient/doctors	GET	Patient	Browse doctors with availability
/api/patient/appointments	POST	Patient	Book an appointment
/api/patient/appointments/<id>/cancel	PUT	Patient	Cancel a booked appointment
/api/patient/appointments/<id>/reschedule	PUT	Patient	Reschedule an appointment
/api/patient/appointments	GET	Patient	View full appointment history
/api/patient/export-csv	POST	**Patient**	Trigger async CSV export of treatment history

7. Implemented Features

Admin:

- Dashboard with live stats: total doctors, patients, and appointments (cached with TTL).
- Add, edit, and delete doctor profiles (name, specialization, qualification, fee, bio).
- View all appointments — upcoming and past — with status filtering.
- Search patients by name or email; search doctors by name or specialization.
- Blacklist or reactivate doctor and patient accounts.

Doctor:

- Dashboard with today's and upcoming appointments and assigned patient list.
- Mark appointments as Completed or Cancelled.
- Add treatment records: diagnosis, prescription, notes, and next visit date.
- Set availability for the next 7 days (date and time slots).
- View full treatment history of assigned patients.

Patient:

- Self-registration and profile management (DOB, gender, blood group, address).
- Browse doctor specializations and availability for the coming 7 days.
- Book, cancel, or reschedule appointments with double-booking prevention.
- View appointment history with diagnosis, prescriptions, and doctor notes.
- Trigger async CSV export of full treatment history.

Background Jobs (Celery + Redis):

- Daily Reminder: Checks today's appointments every morning and alerts patients.
- Monthly Report: Generates HTML activity report per doctor on the 1st of each month.

- CSV Export: Patient-triggered async job exporting all treatment records to CSV.

8. Video Presentation

Drive Link: [HMS v2]